4.2.4 C

The procedural programming aspects of *Mathematica* are quite similar to C.

The following operators work essentially the same in *Mathematica* as in C: `!` (`Not`), `++` (`Increment` and `PreIncrement`), `--` (`Decrement` and `PreDecrement`), `<` (`Less`), etc., `==` (`Equal`), `!=` (`Unequal`), `&&` (`And`), `||` (`Or`), `=` (`Set`), `+=` (`AddTo`), `-=` (`SubtractFrom`), `*=` (`TimesBy`), `/=` (`DivideBy`). There are no bitwise operators built in to *Mathematica*.

C	*Mathematica*
f(*x*)	*f*[*x*]
x % *y*	Mod[*x*, *y*]
floor(*x*), ceil(*x*), sqrt(*x*), etc.	Floor[*x*], Ceiling[*x*], Sqrt[*x*], etc.
a[*i*]	a[[*i*]]
/* *text* */	(* *text* *)
if(*predicate*) *t* else *f*	If[*predicate*, *t*, *f*]
switch(*value*) { case a: *va* ; case *b*: *vb* ; default: *vdef* ; }	Switch[a, *va*, *b*, *vb*, _, *vdef*]
while(*predicate*) *statement*	While[*predicate*, *statement*]
for(*start*; *test*; *increment*) *body*	For[*start*, *test*, *increment*, *body*]
continue	Continue[ ]
break	Break[ ]
identifier: *statement*	Label[*identifier*]; *statement*
return *result*	Return[*result*]

Some correspondences between C and *Mathematica*.

The control structures in *Mathematica* are quite similar to those in C. One important difference is that in *Mathematica* there is no distinction between expressions and statements; every object returns a value. As a result, semicolons in *Mathematica* play the role of both semicolons and commas in C. Commas in *Mathematica* are used only as separators for function arguments.

This has the consequence that in the specification of a `For` loop, the roles of `;` and `,` in *Mathematica* and C are *reversed*. The C code `for(i=0, j=1; i<10; i++)` *statement*; becomes the *Mathematica* code `For[ i=0; j=1, i<10, i++,` *statement* `]`. Note that a null condition in a *Mathematica* `For` is interpreted as *false*, whereas in C a null condition is interpreted as *true*.

You can set up blocks of code with their own local variables in *Mathematica* much as you do in C. However, since every *Mathematica* variable can represent any expression, you do not have to declare the "types" of variables in *Mathematica*. `Block[{`x`,` y`,` z `=` z_0`},` *code*`]` represents a block of code with local variables x, y and z. In this case, the variable z is set to have initial value z_0. Notice that in *Mathematica* there is a *comma* after the list that specifies local variables, whereas in C variable declarations end with semicolons.

Another difference between C and *Mathematica* concerns Boolean tests. True and false are represented in *Mathematica* by the symbols `True` and `False`, rather than `1` and `0`. In addition, symbolic expressions like `x==y` may have undetermined truth value; as a result, functions like `If` have an extra element that specifies what to do in this case.

Lists in *Mathematica* are largely analogous to arrays in C. One important difference is that *Mathematica* arrays have index origin `1`, rather than `0`.

You can output *Mathematica* expressions and code in C format using the function `CForm[`*expr*`]`.